Day 138 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 20, Day 6 reference

Watch: What is Fréchet Inception Distance (FID)? (Data Science Made Easy) | Frechet Inception Distance and Inception Score - AI Bits and Pieces (Shayan Fazeli)

`day138_evaluating_generative_models.py`, segment by segment, in the order `main()` uses things. The matrix square root, the Frechet distance, the Inception Score, k-nearest-neighbour precision and recall, the feature network and the Scorer are new; the layers and the training round are Day 136's, AdamGAN is Day 134's, and sqdist and softmax are Days 119 and 116's, all imported unchanged. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: μ, Σ = mean, covariance of features · p(y|x) = class probabilities of one image · p(y) = their average · k = neighbours (3) · Tr = trace

Segment 1 — The scores

sqrtm_psd: a matrix square root by eigen-decomposition

MATH

$$S = V\,diag(w)\,V^T \Rightarrow S^{1/2} = V\,diag(\sqrt{w})\,V^T,\quad (S^{1/2})^2 = S$$

CODE

```
def sqrtm_psd(S):
    w, V = np.linalg.eigh(S)
    return (V * np.sqrt(np.clip(w, 0, None))) @ V.T
```

- `np.linalg.eigh(S)` — eigenvalues and eigenvectors of a symmetric matrix (eigh is the symmetric version: real results, sorted).
- `np.clip(w, 0, None)` — no lower values than 0, no upper limit: rounding can leave tiny negative eigenvalues.
- `V * np.sqrt(w)` — broadcasting multiplies column j of V by $\sqrt{w_j}$: the same as V @ diag(√w), without building the diagonal matrix.

frechet: the distance between two Gaussians

MATH

$$FID = |\mu_1 - \mu_2|^2 + Tr(\Sigma_1) + Tr(\Sigma_2) - 2\,Tr((\Sigma_1\Sigma_2)^{1/2})$$

CODE

```
def frechet(mu1, S1, mu2, S2):
    A = sqrtm_psd(S1); M = A @ S2 @ A
    tr_sqrt = np.sqrt(np.clip(np.linalg.eigvalsh((M + M.T) / 2), 0, None)).sum()
    return float(((mu1 - mu2) ** 2).sum() + np.trace(S1) + np.trace(S2) - 2 * tr_sqrt)
```

- `A @ S2 @ A` — $S1^{1/2}\,S2\,S1^{1/2}$: symmetric, with the same eigenvalues as S1 S2, so eigvalsh can be used.
- `(M + M.T) / 2` — makes M exactly symmetric; rounding can leave it slightly off.
- `np.linalg.eigvalsh(...)` — eigenvalues only, of a symmetric matrix; their square roots summed give the trace of the square root.

gauss and inception_score

MATH

$$IS = exp(mean_x \Sigma_y p(y|x) [log\ p(y|x) - log\ p(y)]), \quad 1 \le IS \le number\ of\ classes$$

CODE

```
def gauss(F):
    F = F.astype(np.float64); return F.mean(0), np.cov(F, rowvar=False)
def inception_score(P):
    P = np.clip(P.astype(np.float64), 1e-12, 1); py = P.mean(0)
    return float(np.exp((P * (np.log(P) - np.log(py))).sum(1).mean()))
```

- `np.cov(F, rowvar=False)` — rowvar=False: each column is a variable, each row an observation (numpy's default is the other way round).
- `np.clip(P, 1e-12, 1)` — no zero probabilities, so the logarithm stays finite.
- `P * (np.log(P) - np.log(py))` — py broadcasts across rows: each image's KL divergence term by term; .sum(1) adds the ten classes.

precision_recall: neighbourhood balls

MATH

$$precision = share\ of\ fakes\ inside\ some\ real\ point's\ k\text{-}NN\ ball; \quad recall = share\ of\ reals\ inside\ some\ fake's\ ball$$

CODE

```
def precision_recall(R, F, k=3):
    DRR, DFF, DRF = d119.sqdist(R, R), d119.sqdist(F, F), d119.sqdist(R, F)
    rR = np.sort(DRR, 1)[:, k]; rF = np.sort(DFF, 1)[:, k]
    precision = float(np.mean((DRF <= rR[:, None]).any(0)))
    recall = float(np.mean((DRF <= rF[None, :]).any(1)))
    return precision, recall
```

- `np.sort(DRR, 1)[:, k]` — sort each row of distances; column 0 is the point itself (distance 0), so column k is the k-th neighbour.
- `DRF <= rR[:, None]` — rR[:, None] makes a column: each real point's radius compared against every fake.
- `.any(0)` — down the columns: does ANY real ball contain this fake?
- `.any(1)` — along the rows: does ANY fake ball contain this real image?

Segment 2 — The feature network

feature_net: a small convolutional classifier

CODE

```
def feature_net(seed=0):
    r = np.random.RandomState(seed)
    body = d136.Net([d136.Act((-1, 1, 28, 28)), d136.Conv(1, 16, 4, 2, 1, r), d136.Act("lrelu"),
        d136.Conv(16, 32, 4, 2, 1, r),
                     d136.Act("lrelu"), d136.Act((-1, 32 * 49)), d136.Dense(32 * 49, 64, r),
                         d136.Act("relu")])
    head = d136.Net([d136.Dense(64, 10, r)])
    return body, head
```

- `d136.Act((-1, 1, 28, 28))` — Day 136's reshape layer: flat rows of 784 back into one-channel images.

- `d136.Conv(1, 16, 4, 2, 1, r)` — 1 channel in, 16 out, kernel 4, stride 2, padding 1: 28 × 28 becomes 14 × 14.
- `body, head` — two stacks, so the 64 features (body) can be read separately from the class scores (head).

train_classifier: softmax cross-entropy

MATH

$$d\ loss\ /\ d\ scores\ =\ (softmax(scores)\ –\ onehot(y))\ /\ batch$$

CODE

```
def train_classifier(X, y, epochs=3, batch=128, seed=0):
    body, head = feature_net(seed); ob, oh = d134.AdamGAN(body.net, 1e-3, 0.9),
        d134.AdamGAN(head.net, 1e-3, 0.9)
    rng = np.random.RandomState(seed); Y = np.eye(10, dtype=np.float32)[y]
    for ep in range(epochs):
        o = rng.permutation(len(X))
        for s in range(0, len(X) - batch + 1, batch):
            b = o[s:s + batch]; L = head.forward(body.forward(X[b]))
            dL = (d116.softmax(L) - Y[b]) / batch
            gh, dF = head.backward(dL); gb, _ = body.backward(dF)
            oh.step(head.net, gh); ob.step(body.net, gb)
    return body, head
```

- `np.eye(10, dtype=np.float32)[y]` — row y of the identity matrix: every label as a one-hot row at once.
- `(d116.softmax(L) - Y[b]) / batch` — the gradient of the average cross-entropy with respect to the scores.
- `gh, dF = head.backward(dL); gb, _ = body.backward(dF)` — the head's input gradient becomes the body's output gradient.

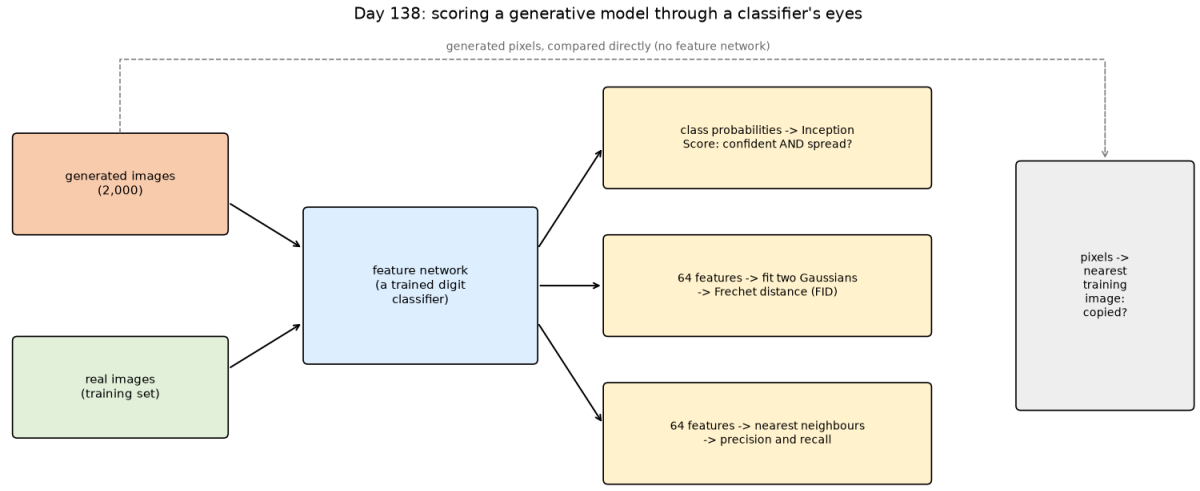
Scorer: every score of one set of images

CODE

```
class Scorer:
    def __init__(self, body, head, Xref, Xnear):
        self.body, self.head = body, head
        self.Fref = self.feats(Xref); self.mu, self.S = gauss(self.Fref)
        self.Fpr = self.Fref[:2000]; self.Xnear = Xnear
    def feats(self, X):
        return np.concatenate([self.body.forward(X[i:i + 1000]) for i in range(0, len(X), 1000)])
    def __call__(self, X):
        F = self.feats(X); P = d116.softmax(self.head.forward(F).astype(np.float64)); mu, S =
            gauss(F)
        p, r = precision_recall(self.Fpr, F[:2000])
        near = float(np.sqrt(np.maximum(d119.sqdist(X[:500], self.Xnear).min(1), 0)).mean())
        return inception_score(P), frechet(mu, S, self.mu, self.S), p, r, near
```

- `class Scorer:` — a class keeps the real references (features, mean, covariance) computed once, for every set scored later.
- `def __call__(self, X):` — makes an instance callable like a function: sc(images).
- `[self.body.forward(X[i:i + 1000]) for i in range(0, len(X), 1000)]` — 1,000 images at a time, to keep memory down; concatenate joins the pieces.
- `F[:2000]` — precision and recall always use 2,000 images, because the ball radii depend on how many points there are.

SYSTEM DIAGRAM



How each score sees the images.

Segment 3 — The experiments

demo_real: the test set in a random order, and in its stored one

CODE

```
perm = 60000 + np.random.RandomState(0).permutation(10000)
rows, sizes = [], (500, 1000, 2000, 5000, 10000)
print(f"{'set scored':>40} {'IS':>7} {'FID':>8} {'precision':>10} {'recall':>7} {'nearest':>8}")
for n in sizes:
    s = sc(X[perm[:n]]); rows.append((n, s)); print(f"{f'{n:,} real test images, random
        order':>40} {s[0]:7.3f} {s[1]:8.3f} {s[2]:10.3f} {s[3]:7.3f} {s[4]:8.2f}")
print("FID falls as the scored set grows, with nothing about the images changing: a fitted
    covariance from few samples is noisy,")
print("and the noise adds to the distance. Scores are only comparable at the same sample size;
    2,000 is used below")
pre = [fid_only(sc, X[60000:60000 + n]) for n in sizes]
```

- `60000 + np.random.RandomState(0).permutation(10000)` — the numbers 0–9,999 shuffled, shifted to the test images' positions 60,000–69,999.
- `X[perm[:n]]` — fancy indexing: the first n positions of the shuffled list, so every size is a random sample.
- `rows.append((n, s))` — keep each size with its five scores, for the figure.
- `X[60000:60000 + n]` — the first n test images in the order MNIST stores them, which section 3 shows is not random.

blur: a 3 × 3 box blur

CODE

```
def blur(X):
    I = np.pad(X.reshape(-1, 28, 28), ((0, 0), (1, 1), (1, 1)), mode="edge")
    return sum(I[:, i:i + 28, j:j + 28] for i in range(3) for j in range(3)).reshape(-1, 784) / 9
```

- `np.pad(..., mode="edge")` — add a one-pixel border copied from the nearest edge pixel.
- `sum(I[:, i:i + 28, j:j + 28] for i ... for j ...)` — a generator expression over the nine shifts: each pixel plus its eight neighbours.

demo_damage: the damaged sets

CODE

```
rng = np.random.RandomState(0); T, yt = X[perm], y[perm]; base = T[:2000]
low = T[yt < 5][:2000]
one = np.concatenate([T[yt == d][:1].repeat(200, 0) for d in range(10)])
sets = [("real test images", base), ("+ noise, sd 0.1", np.clip(base + 0.1 *
    rng.randn(*base.shape), 0, 1).astype(np.float32)),
        ("+ noise, sd 0.3", np.clip(base + 0.3 * rng.randn(*base.shape), 0,
            1).astype(np.float32)), ("3 x 3 blur", blur(base).astype(np.float32)),
        ("digits 0-4 only", low), ("one image per digit, x 200", one), ("copies of training
            images", X[10000:12000])]
```

- `T, yt = X[perm], y[perm]` — the images and their labels reordered together, with section 3's shuffle.
- `T[yt < 5]` — a boolean mask: only the images whose label is below 5.
- `T[yt == d][:1].repeat(200, 0)` — the first image of digit d, repeated 200 times along axis 0.
- `rng.randn(*base.shape)` — * unpacks the shape tuple into separate arguments: randn(2000, 784).

train_scored: Day 134's loop, scored as it goes

CODE

```
def train_scored(G, D, X, sc, b1, epochs, seed, at, k=64, batch=128):
    oG, oD = d134.AdamGAN(G.net, 2e-4, b1), d134.AdamGAN(D.net, 2e-4, b1)
    rng = np.random.RandomState(seed); Zfix = np.random.RandomState(9).standard_normal((2000,
        k)).astype(np.float32)
    log, t0 = [], time.perf_counter()
    for ep in range(1, epochs + 1):
        o = rng.permutation(len(X))
        for s in range(0, len(X) - batch + 1, batch):
            d136.net_gan_step(G, D, oG, oD, X[o[s:s + batch]], rng, k)
        if ep in at:
            F = d136.net_generate(G, Zfix); log.append((ep, sc(F), F[:16].copy(),
                time.perf_counter() - t0))
    return log
```

- `d136.net_gan_step(G, D, oG, oD, X[o[s:s + batch]], rng, k)` — one round of the game, Day 136's version for layer stacks.
- `if ep in at:` — score only at the chosen epochs: scoring costs a pass over 2,000 images and a nearest-image search.
- `F[:16].copy()` — keep 16 images for the figure.

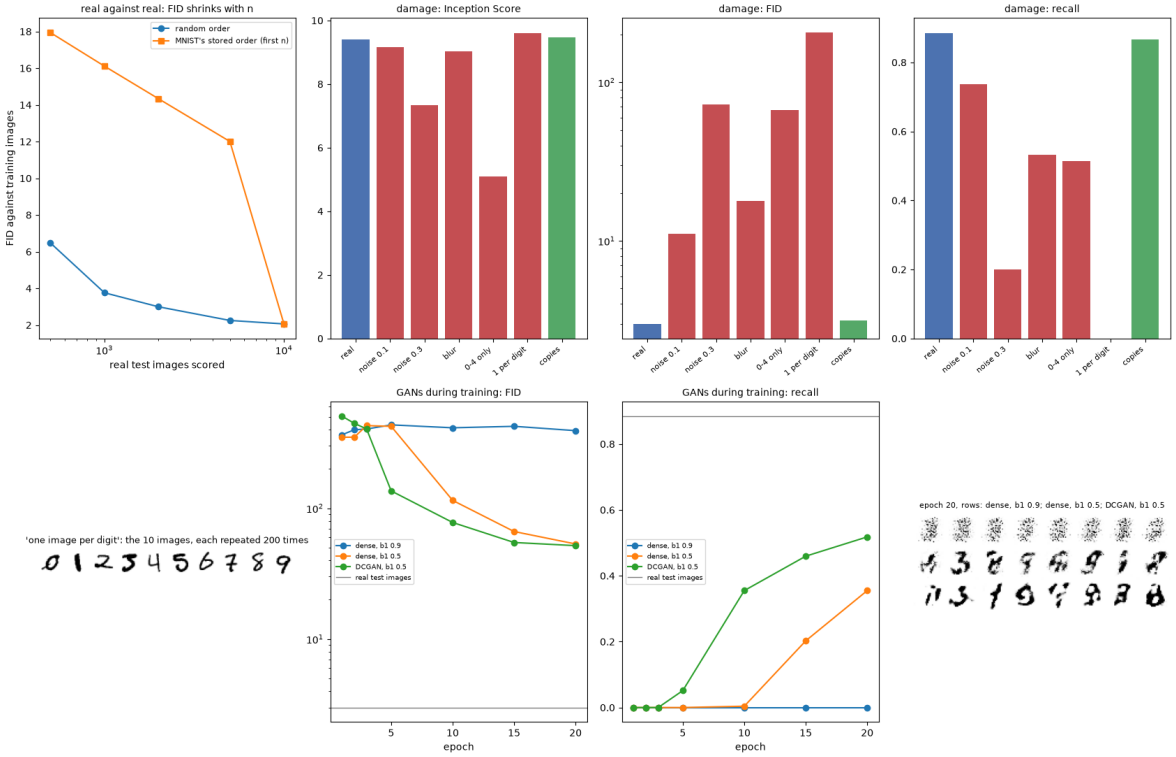
demo_gans: three GANs

CODE

```
for name, b1, maker in (("dense, b1 0.9", 0.9, d136m.dense_players), ("dense, b1 0.5", 0.5,
    d136m.dense_players), ("DCGAN, b1 0.5", 0.5, d136.dcgan_players)):
    G, D = maker(0); log = train_scored(G, D, X[:20000], sc, b1, epochs, 0, at); runs[name] = log
```

- `for name, b1, maker in (...)` — unpacks each triple; maker is a function (which players to build), called as maker(0).
- `runs[name] = log` — a dictionary keeps the three histories under their names, for the table and the figure.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.linalg.eigh / eigvalsh` — eigen-decomposition of a symmetric matrix / its eigenvalues only.
- `np.cov(F, rowvar=False)` — covariance with variables in columns.
- `A[:, None]` — add an axis: a row of values becomes a column, for broadcasting.
- `m.any(0) / m.any(1)` — is any entry true, down each column / along each row.
- `def __call__(self, ...)` — lets an object be called like a function.
- `f(*shape)` — unpack a tuple into separate arguments.

End of Day 138 syntax reference. For the experiments and what they showed, see the Day 138 complete guide.